

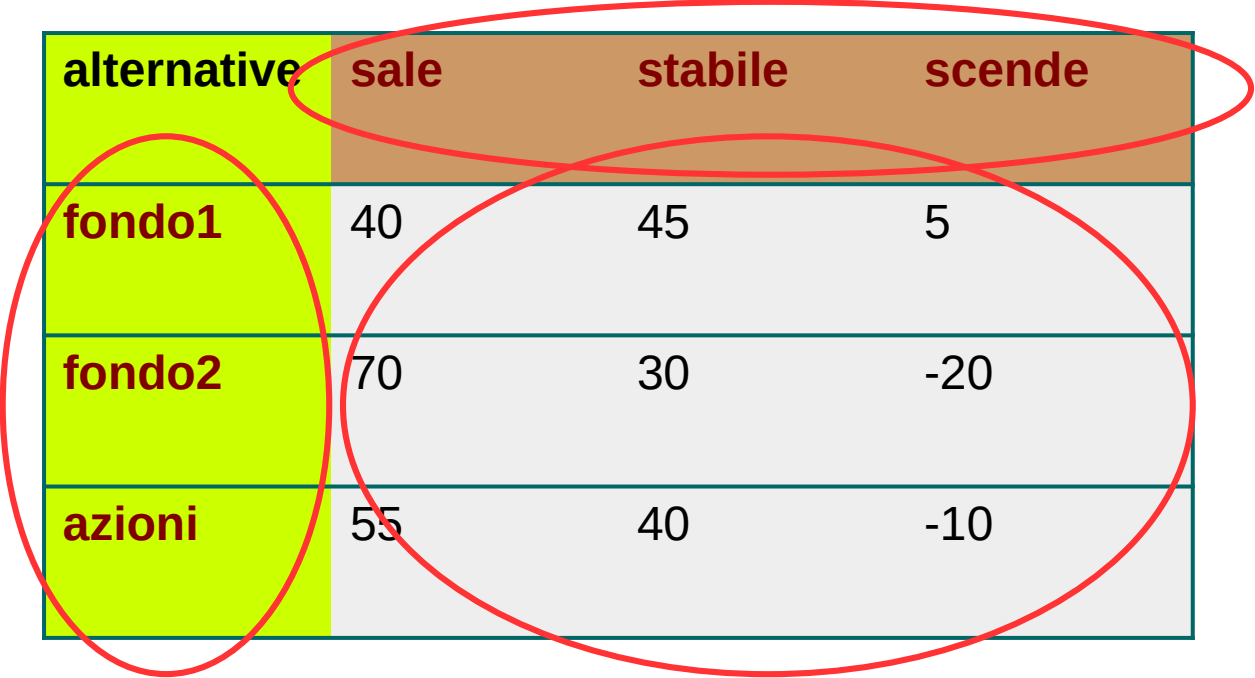
Approcci maximax, maximin, minimax regret (nella teoria delle decisioni)

- Approccio maximax (ottimistico)
- Approccio maximin (pessimistico o conservativo)
- Approccio minimax regret (“pentimento” minimax)
- Esempio: occorre scegliere quale investimento fare. È data una tabella dei payoff

alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Approcci maximax, maximin, minimax regret (teoria delle decisioni)

Andamenti: non sono controllabili,
dipendono da dinamiche esterne



alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Scelte, se ne può fare
una sola

Payoff: possono essere guadagno o
perdite ma anche riferirsi ad altre misure
(esempio: tempo, risorse). Sono specifici
del problema

Approccio maximax

Guarda i payoff più alti per ogni possibile scelta e fa la scelta che promette di più in assoluto (fondo2 dà 70): **il massimo dei massimi**. E' ottimistica perché non ha garanzie che le dinamiche esterne faranno salire il fondo scelto (nell'esempio).

alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Approccio maximin

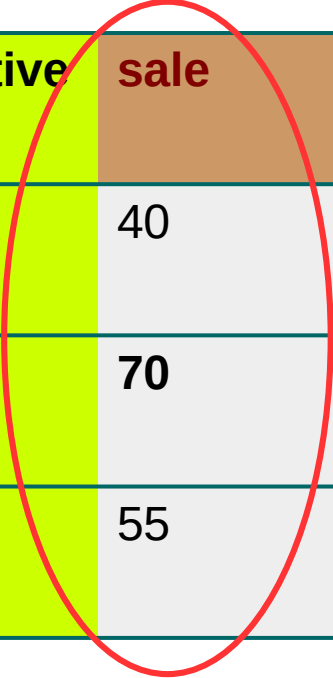
Questo approccio guarda le perdite maggiori legate a ciascuna scelta e poi esegue l'azione che minimizza le perdite (**il massimo dei minimi**). Nell'esempio sceglierà fondo 1 che è l'unica opzione a non andare in perdita.

alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Approccio minimax regret: regret

Best regret = Best payoff – Real payoff

Se il mercato sale il best payoff è 70. Se abbiamo investito in fondo1 il real regret è 70-40 Cioè 30. Se abbiamo investito in azioni è 15 e se abbiamo investito in fondo2 è 0.



alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Approccio minimax regret: regret

Best regret = Best payoff – Real payoff

Stesso genere di conti viene fatto per gli altri casi: mercato stabile o in discesa.



alternative	sale	stabile	scende
fondo1	40	45	5
fondo2	70	30	-20
azioni	55	40	-10

Si calcola una **regret table** ...

Approccio minimax regret: regret table

Si calcolano i massimi regret per ogni scelta alternativa e poi si seleziona l'alternativa che porta al regret minimo, nell'esempio acquistare azioni che dà un pentimento di 15.

alternative	sale	stabile	scende
fondo1	30	0	0
fondo2	0	15	25
azioni	15	5	15

Leggendo le righe: fondo1 dà regret massimo pari a 30, fondo2 dà 25, azioni dà 15.

Guadagno e giochi con avversario

- Nel nostro caso le dinamiche dell'ambiente sono costituite dall'avversario:
 - Come non sappiamo come evolverà l'ambiente, non sappiamo quale scelta farà l'avversario
 - Dobbiamo far bastare la *conoscenza dello stato corrente* e delle *mosse a disposizione*

Caratteristiche del gioco

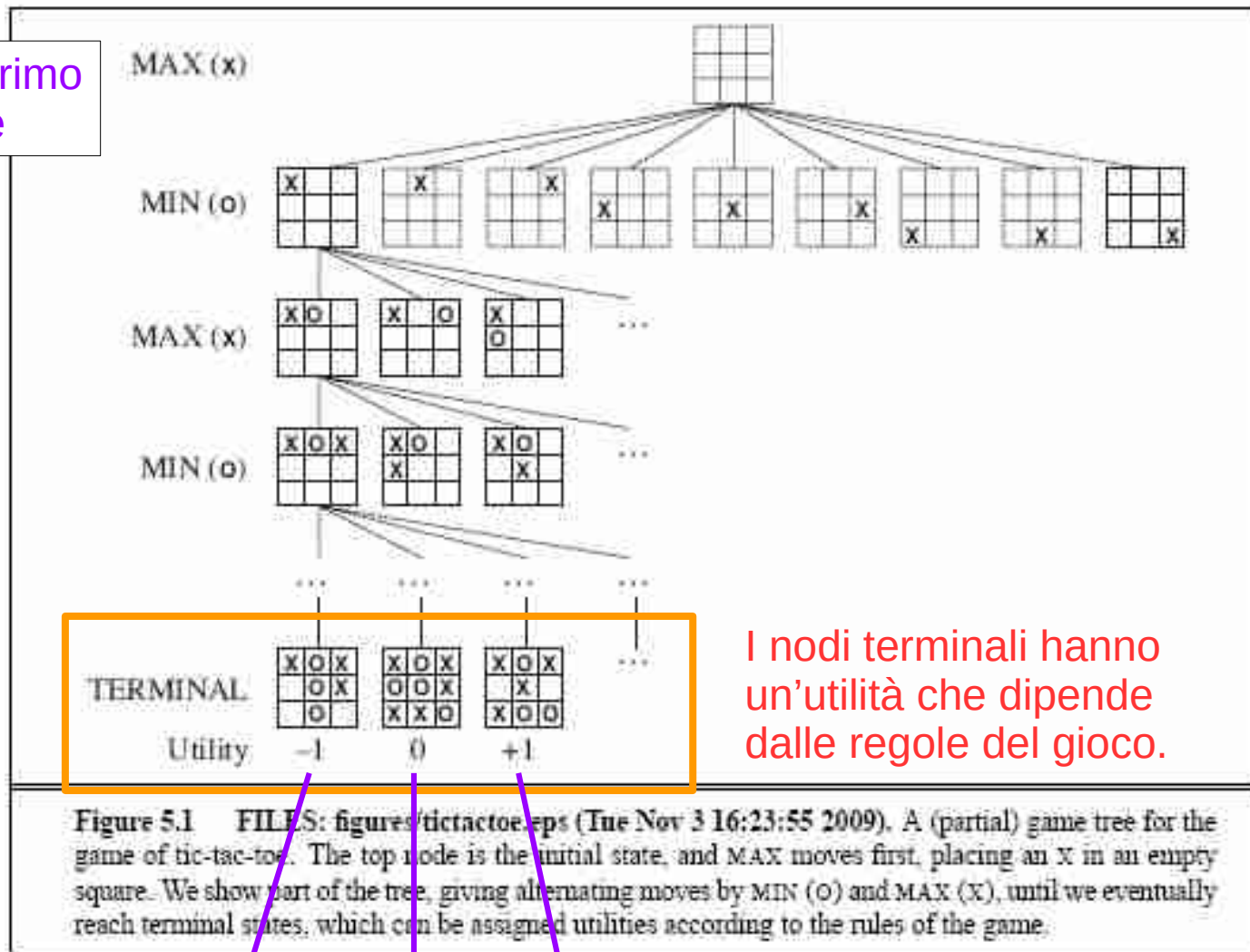
- **Due giocatori**, può essere comodo chiamarli MAX e MIN con **MAX che muove per primo**
- Ciascun giocatore non sa quali mosse farà l'altro ma le mosse possibili sono note e sono calcolabili i **successori** che produrranno una volta applicate a qualche stato
- **Osservabilità:**
 - È **totale** nel caso di giochi con turno: i giocatori conoscono i risultati delle mosse precedenti
 - È **parziale** nel caso di giochi ad azione simultanea: i giocatori non conoscono le mosse che i giocatori eseguono simultaneamente alla loro
- Partendo da uno **stato iniziale** è possibile sviluppare un **albero di possibili evoluzioni (albero di gioco)**, applicando le azioni eseguibili e calcolando così gli stati successivi
- Alcuni stati sono **terminali**, quando uno di essi è raggiunto la partita termina

Caratteristiche del gioco

- I giocatori si avvalgono del calcolo dell'**utilità degli stati**
- I giocatori devono **tener conto dell'avversario** quindi il calcolo dell'utilità comprende una valutazione del punto di vista dell'avversario
- **Giocatori pessimisti**: suppongono che l'avversario faccia sempre la mossa che gli porta il guadagno maggiore
- **Giochi con turno (2-ply)**

Utilità soggettiva nel caso del tris

MAX è il primo a muovere



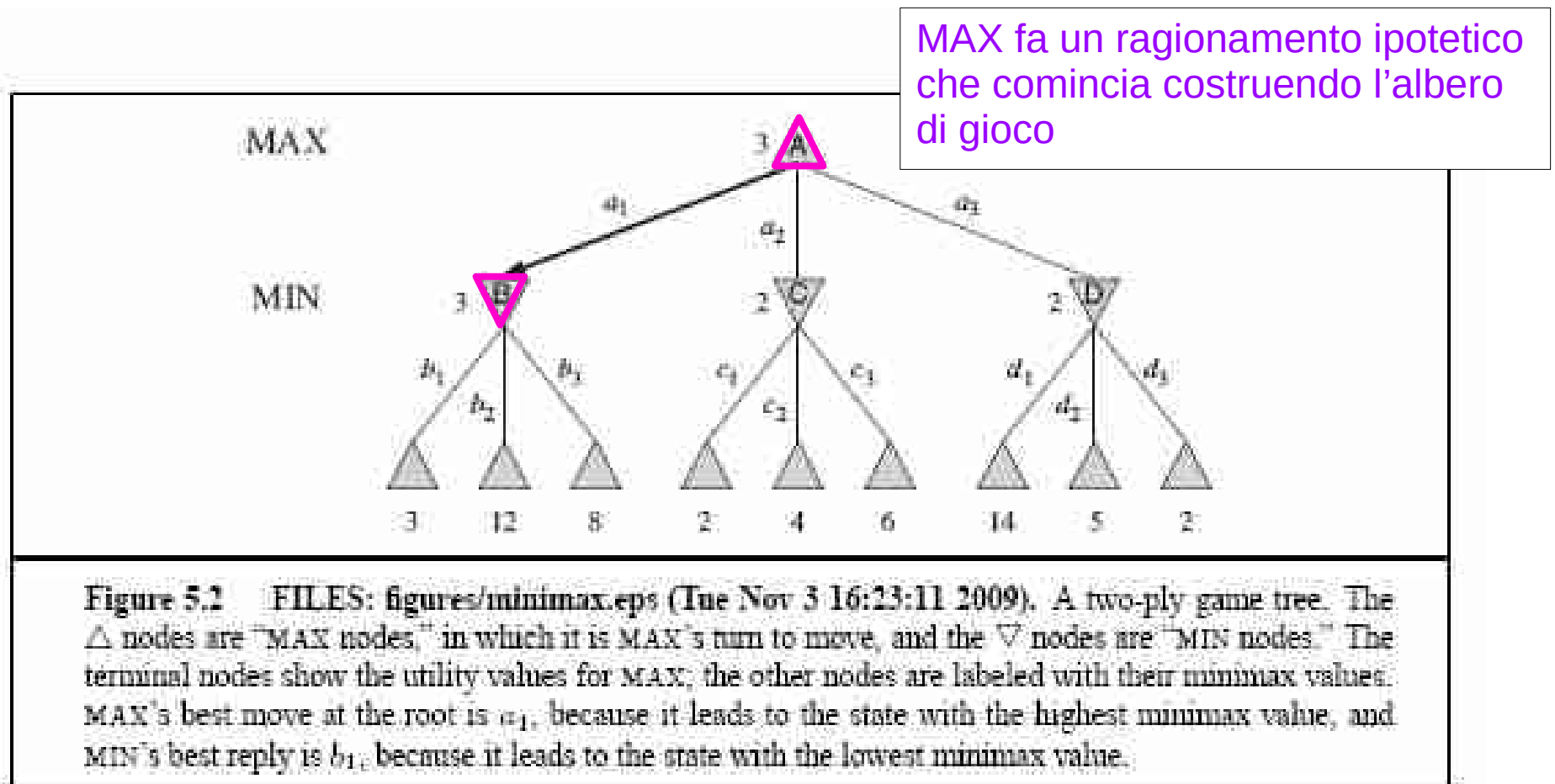
I nodi terminali hanno un'utilità che dipende dalle regole del gioco.

Cattivo, neutro, buono per MAX

Strategia ottima per un agente

- Sequenza di mosse che porta a uno stato terminale corrispondente alla vittoria dell'agente
- L'agente non sa come muoverà l'altro, può solo "immedesimarsi"

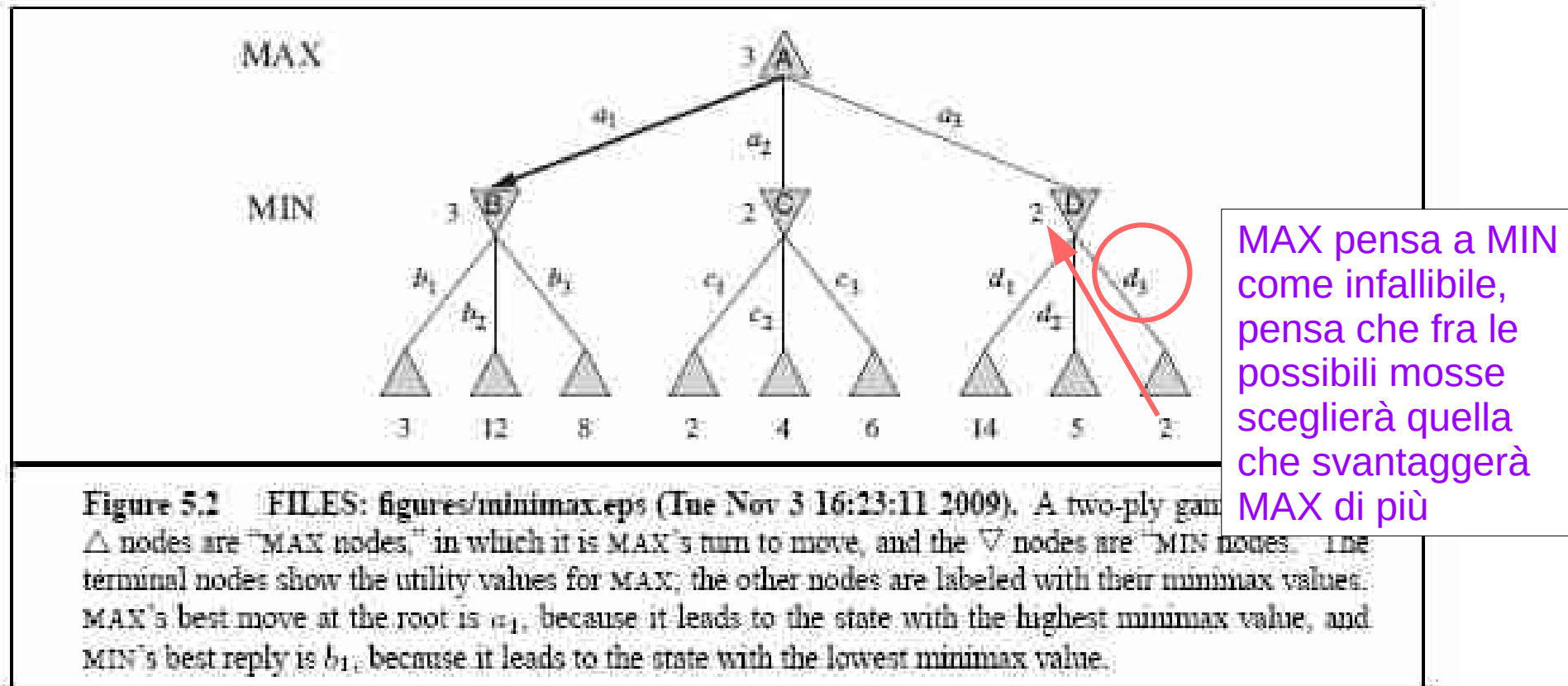
Ricerca strategia ottima, esempio



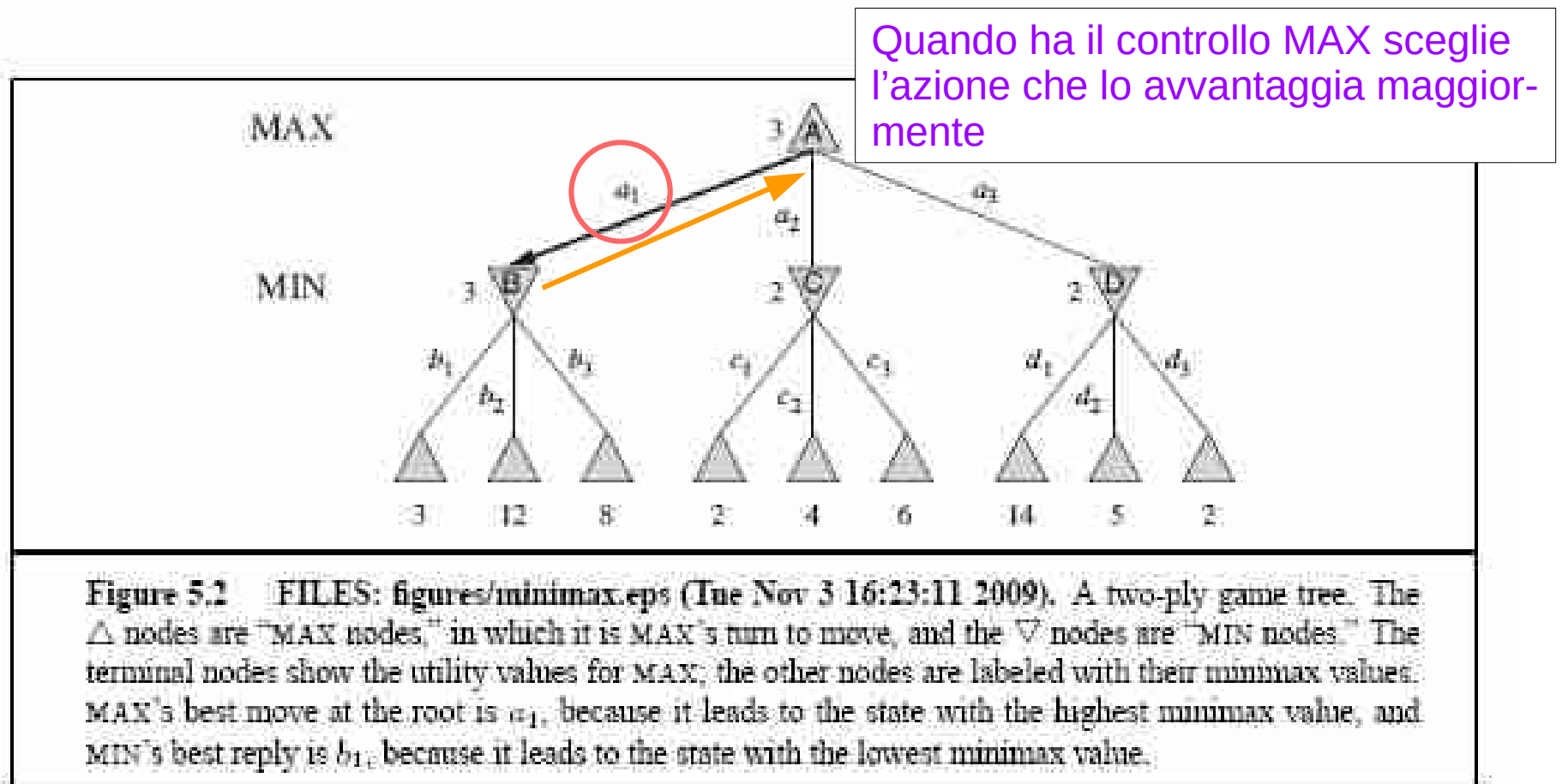
$\triangle$ Nodi MAX

∇ Nodi MIN

Ricerca strategia ottima, esempio



Ricerca strategia ottima, esempio



Valore-minimax(n)

$$\text{Valore-minimax}(n) = \begin{cases} \text{utilità}(n) & \text{se } n \text{ terminale} \\ \max_{s \in \text{succ}(n)} \text{valore-minimax}(s) & \text{se } n \text{ è un nodo MAX} \\ \min_{s \in \text{succ}(n)} \text{valore-minimax}(s) & \text{se } n \text{ è un nodo MIN} \end{cases}$$

È un valore calcolato per ogni nodo. Permette all'agente la scelta della mossa da eseguire. I nodi MAX e MIN rappresentano due tipi di scelte: MAX è una **scelta interna** dell'agente mentre MIN è una **scelta esterna**, cioè tale che l'agente non ha controllo su di essa

Il nome minimax deriva dal fatto che ogni giocatore, minimizzando il guadagno massimo dell'altro minimizza la propria perdita

Nei **giochi a due** l'utilità per un giocatore è uguale a meno l'utilità per l'altro giocatore. Basta ricordare uno di questi valori per avere automaticamente anche l'altro

Algoritmo minimax ricorsivo

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
```

```
   $v \leftarrow \text{MAX-VALUE}(\textit{state})$ 
  return the action in SUCCESSORS(state) with value  $v$ 
```

```
function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
  return  $v$ 
```

```
function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow \infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
  return  $v$ 
```

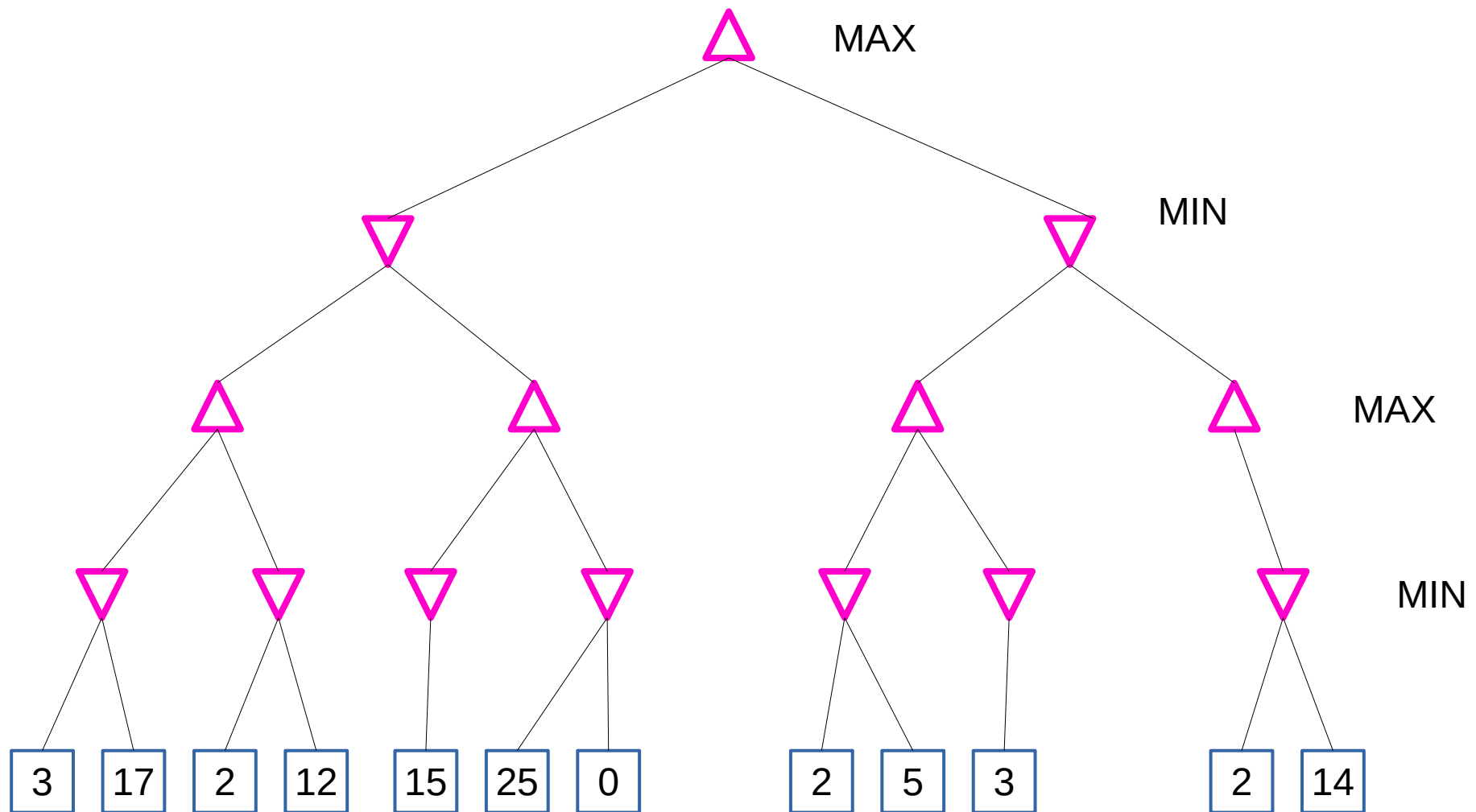
Figure 6.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state.

Visita completa in profondità dell'albero di gioco

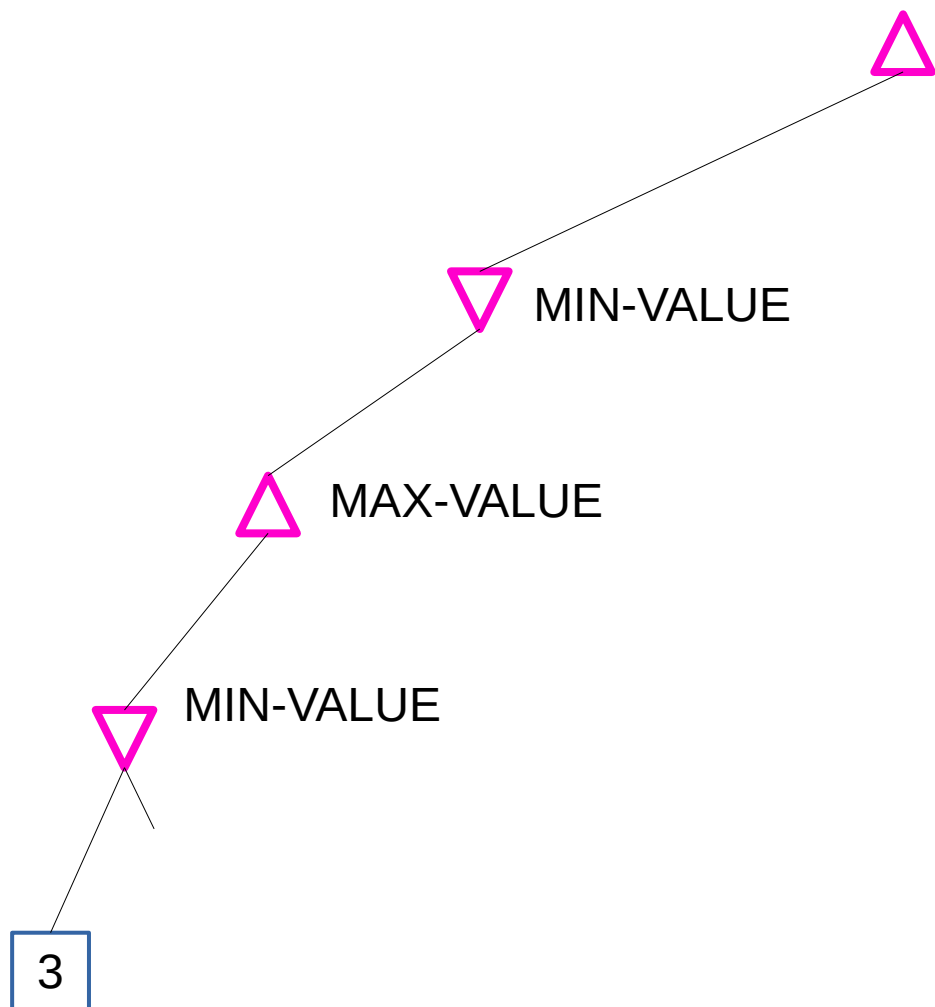
Complessità temporale: **$O(b^m)$**
con b = branching factor ed
 m = profondità massima

Complessità spaziale: **$O(bm)$**
se i successori sono generati
contemporaneamente

Esempio: albero di gioco completo



Esempio: albero di gioco completo



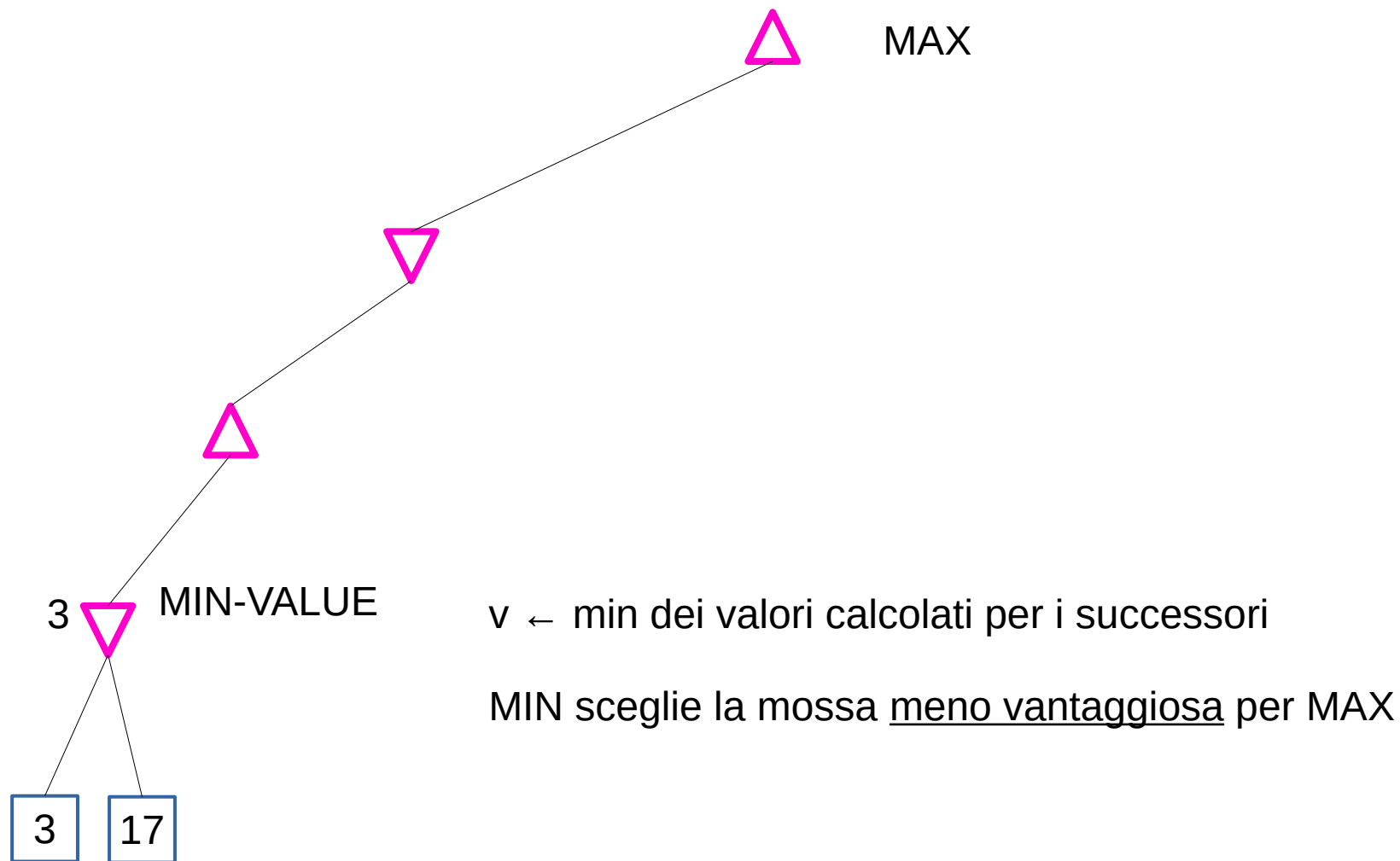
MAX-VALUE

Esplorazione ricorsiva dell'intero albero con chiamate alternate a max-value e a min-value.

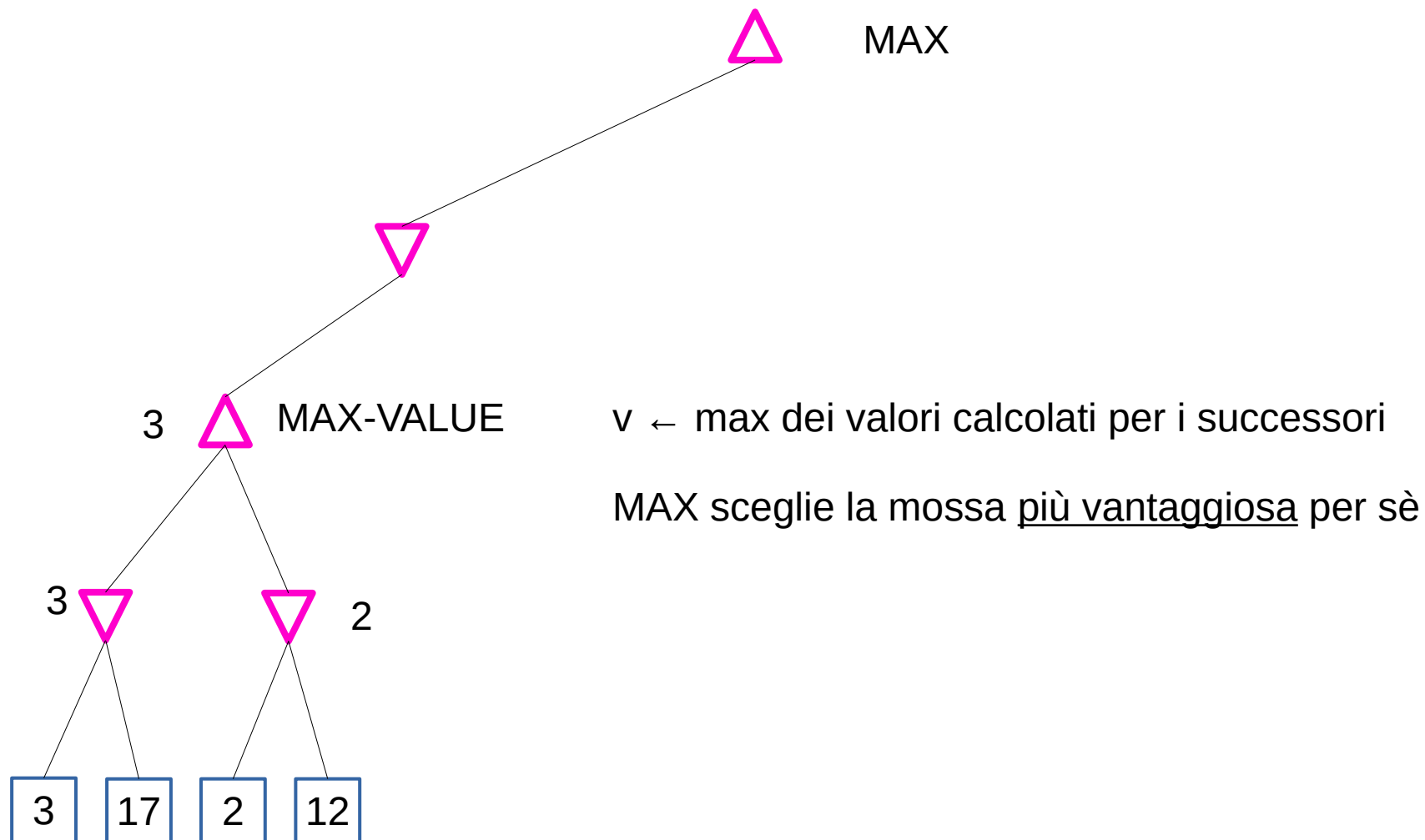
I nodi assumeranno un valore alla chiusura delle rispettive chiamate ricorsive.

I nodi terminali hanno come valore la loro utilità.

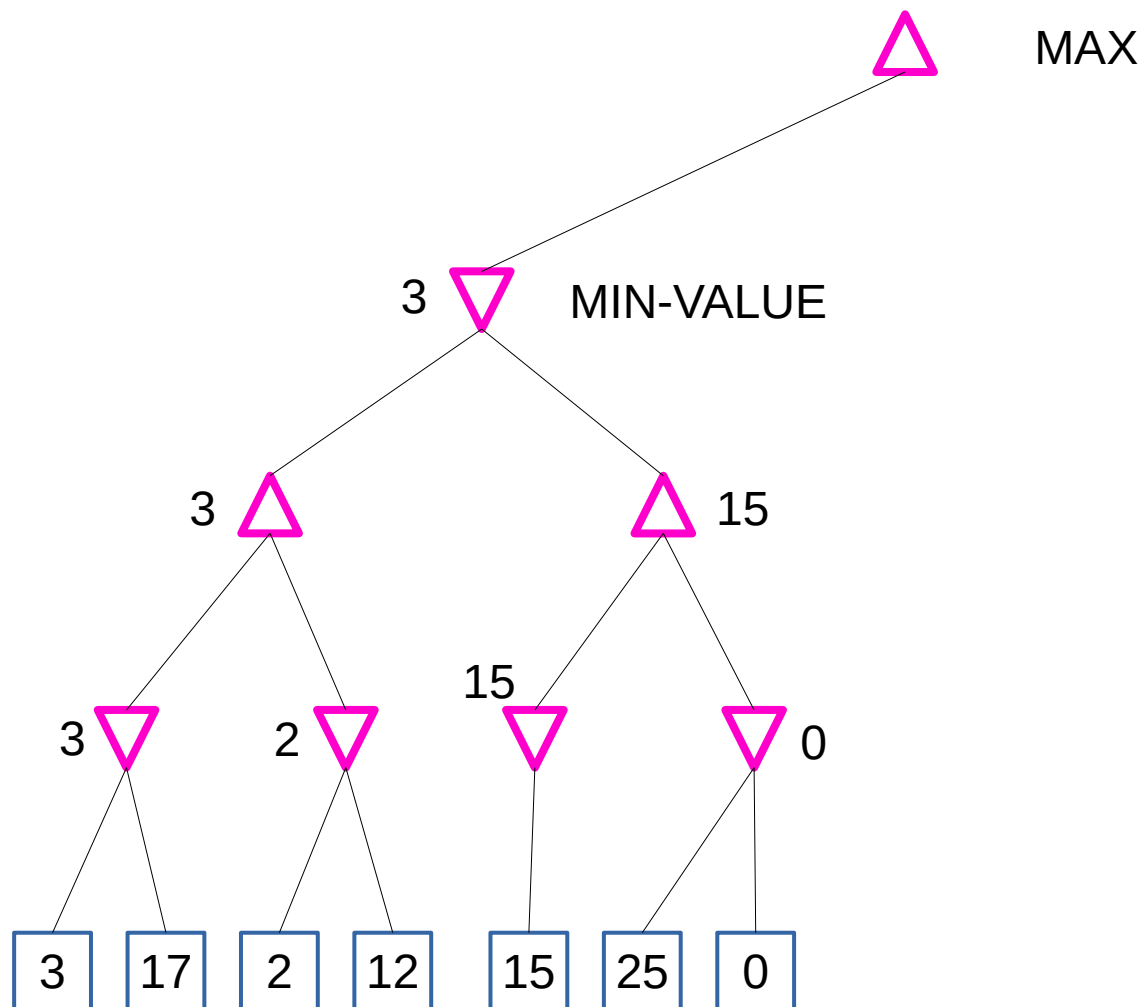
Esempio: albero di gioco completo



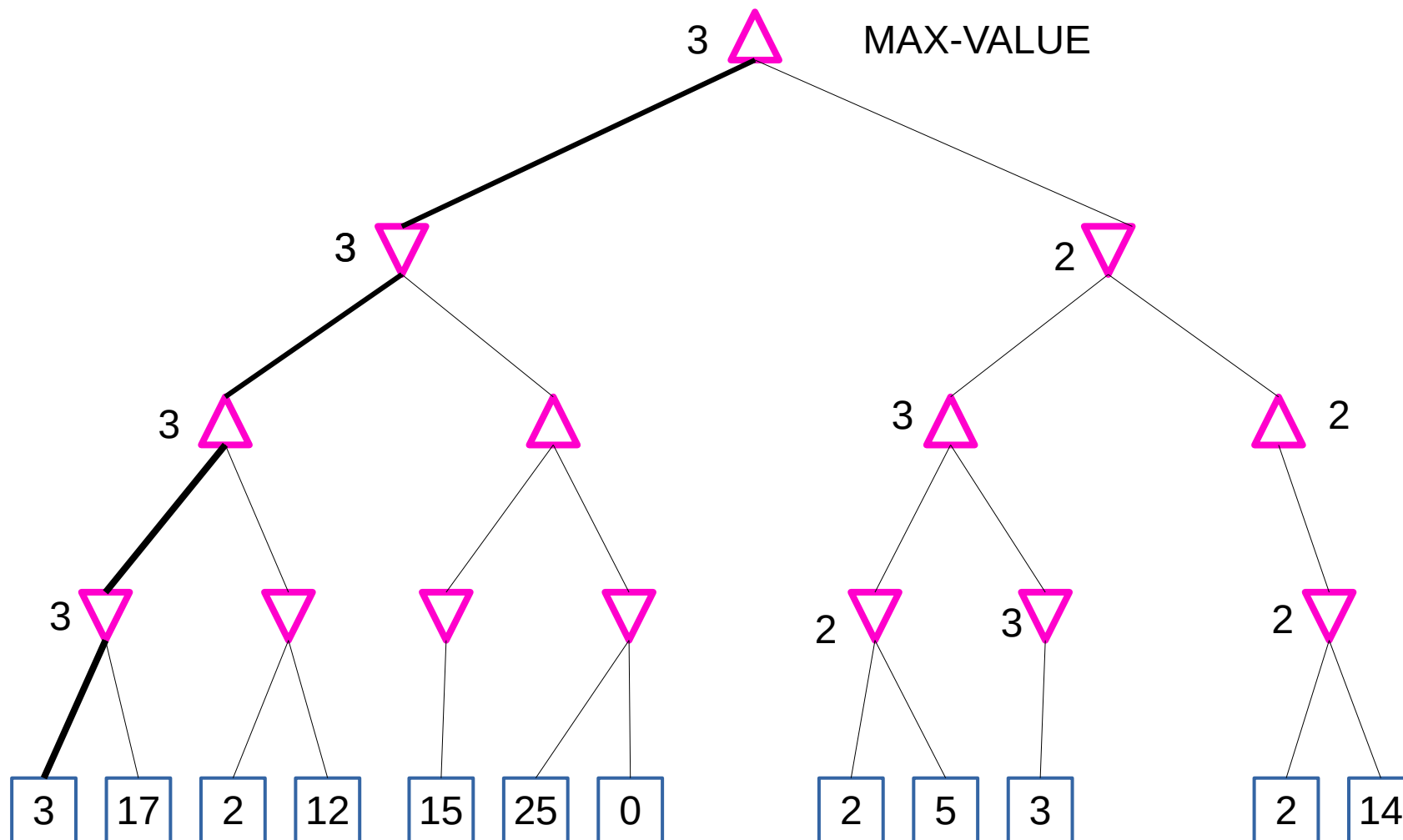
Esempio: albero di gioco completo



Esempio: albero di gioco completo



Esempio: albero di gioco completo



Giochi con più di due giocatori

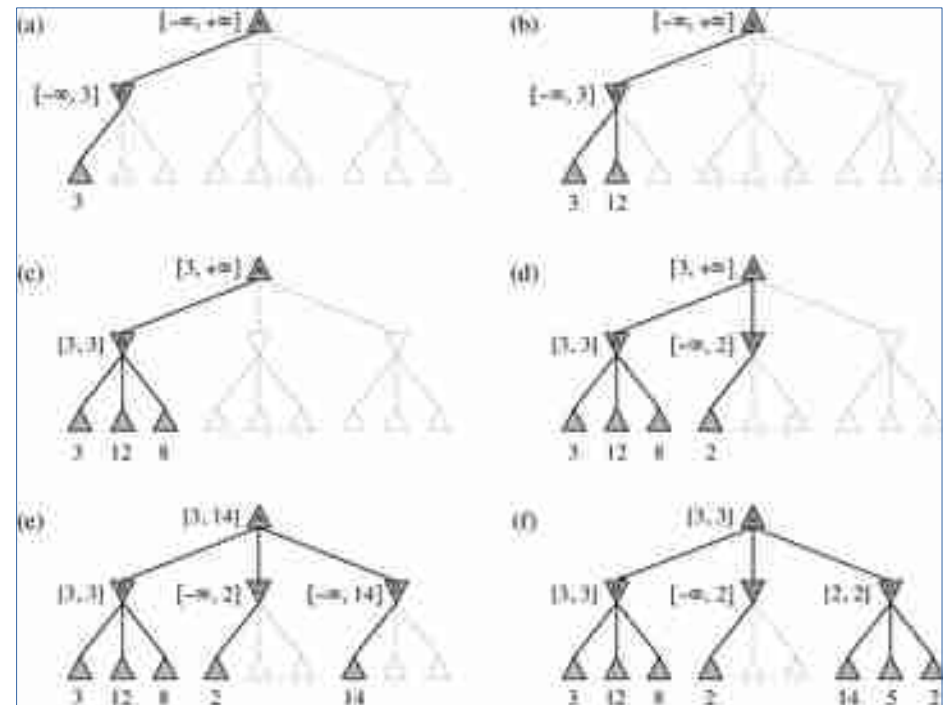
- Minimax può essere esteso a **più giocatori**
- Utilità di ogni nodo data da un vettore $\langle u_1, \dots, u_k \rangle$ dove u_i indica l'utilità del nodo per il giocatore i
- Un aspetto dei giochi a più di due giocatori è la formazione di alleanze

- Effettua una visita in profondità completa quindi la complessità temporale è esponenziale e quella spaziale è lineare
- È completo in grafi finiti
- È ottimale se MAX e MIN giocano in modo ottimale

Minimax con potatura Alfa-Beta

- In termini di tempo, minimax è molto costoso
- Per ridurre i tempi di ricerca: potare i rami meno promettenti senza seguirli
- Alfa-beta è un algoritmo di potatura

nell'esempio le foglie figlie del nodo centrale non espresse hanno valutazioni più alte di 2. È inutile considerarle perché darebbero un vantaggio all'avversario.



Potatura alfa-beta

- Ogni nodo ha associato un valore N che cambia man mano che l'esplorazione dei suoi sottoalberi procede
- L'esplorazione si porta dietro (e aggiorna) gli estremi di un intervallo $[\alpha, \beta]$:
 - α = **massimo lower bound** delle soluzioni possibili (è posto dai nodi MAX)
= valore della scelta migliore per MAX trovata in qualsiasi punto di scelta lungo
il cammino (inizialmente $-\infty$)
 - β = **minimo upper bound** delle soluzioni possibili (è posto dai nodi MIN)
= valore della scelta migliore per MIN trovata in qualsiasi punto di scelta lungo
il cammino (inizialmente ∞)
- Ha senso esplorare un nodo se e solo se il suo valore stimato N è compreso fra α e β

Potatura alfa-beta

- I valori α e β sono aggiornati man mano che la ricerca procede
- Idealmente α e β sono gli estremi di un intervallo che si restringe con la ricerca:

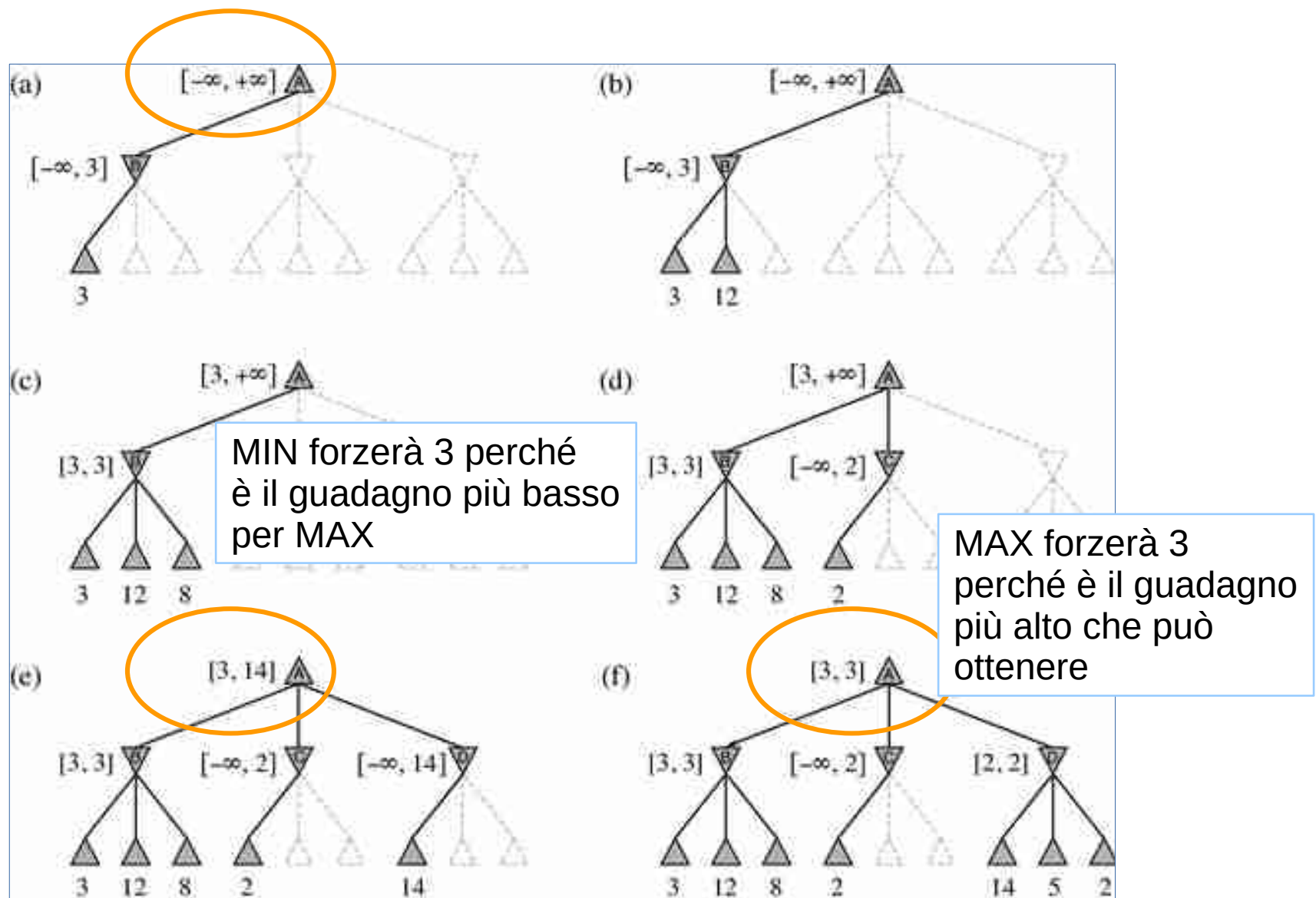


- I nodi che hanno valutazioni fuori dall'intervallo sono esclusi
- Se a un certo punto per un certo nodo i due estremi si invertono, si può evitare di condurre l'esplorazione dei sottoalberi rimanenti di quel nodo



- Nessun valore potrà infatti essere minore di β e maggiore di α

Esempio



Minimax con potatura alfa-beta: algoritmo

function ALPHA-BETA-SEARCH(*state*) **returns** an action

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the action in $\text{SUCCESSORS}(\text{state})$ with value v

function MAX-VALUE(*state*, α , β) **returns** a utility value

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if $\text{TERMINAL-TEST}(\text{state})$ **then return** $\text{UTILITY}(\text{state})$

$v \leftarrow -\infty$

for a, s **in** $\text{SUCCESSORS}(\text{state})$ **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

MAX-VALUE modifica il lower bound

function MIN-VALUE(*state*, α , β) **returns** a utility value

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if $\text{TERMINAL-TEST}(\text{state})$ **then return** $\text{UTILITY}(\text{state})$

$v \leftarrow +\infty$

for a, s **in** $\text{SUCCESSORS}(\text{state})$ **do**

$v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$

if $v < \alpha$ **then return** v

$\beta \leftarrow \text{MIN}(\beta, v)$

return v

MIN-VALUE modifica l'upper bound